How many ways we can fetch OR retrieve the Collection Objects ?

* There are 9 ways to retrieve the Collection Objects which are as follows :

1) By using toString() method of respective class. [JDK 1.0V]
2) By using ordinary for loop. [JDK 1.0V]
3) By using for-each loop. [JDK 1.5V]
4) By using Enumeration<E> interface. [JDK 1.0V]
5) By using Iterator<E> interface. [JDK 1.2V]
6) By using ListIterator<E> interface. [JDK 1.2V]
7) By using forEach(Consumer<T> cons) [JDK 1.8V]
8) By using Spliterator<E> interface. [JDK 1.8V]
9) By using Method Reference. [JDK 1.8V]

Retrieve the Collection Object by using all 9 ways :

Enumeration<E> interface :

It is a predefined interface available in java.util package from JDK 1.0 onwards(Legacy interface).

We can use Enumeration interface to fetch or retrieve the Objects one by one from the Collection because it is a cursor.

We can create Enumeration object by using elements() method of the legacy Collection class. Internally it uses anonymous inner class object without ref.

public Enumeration<E> elements();

Enumeration interface contains two methods :

1) public boolean hasMoreElements() :- It will return true if the collection object is available in the current index position.

2) public E nextElement() :- It will return collection object so return type is Object(E) and move the cursor to the next index.

Note :- It will only work with legacy Collections classes.

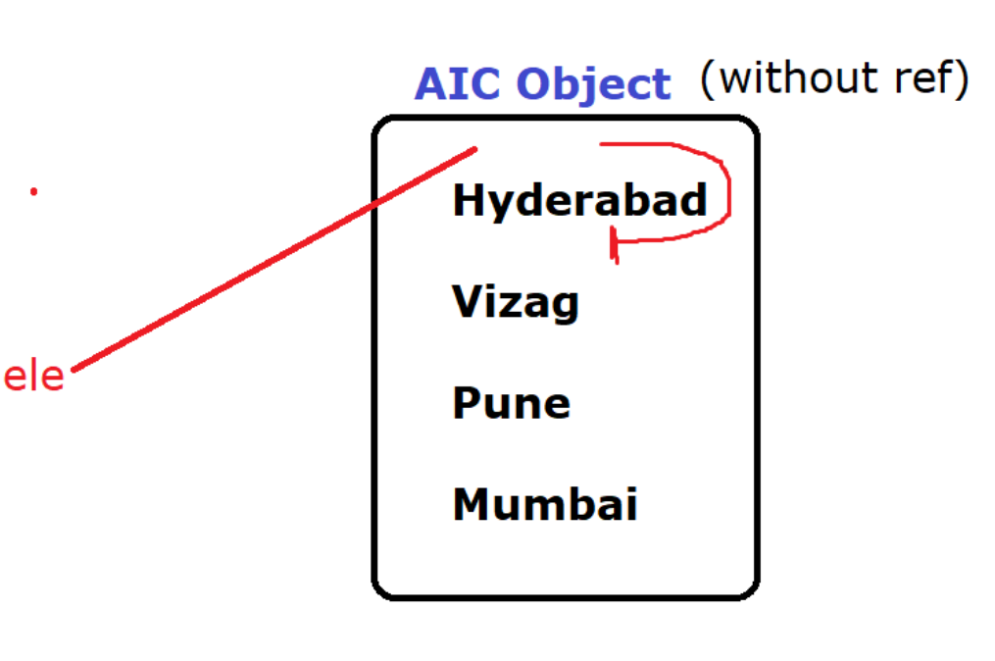
Internal Working :

```
Vector<String> listOfCity = new Vector<>();
      listOfCity.add("Hyderabad");
      listOfCity.add("Vizag");
      listOfCity.add("Pune");
      listOfCity.add("Mumbai");

     Enumeration<String>  ele = listOfCity.elements();

     while(ele.hasMoreElements())
     {
         IO.println(ele.nextElement());

     }
```



Iterator<E>

It is a predefined interface available in java.util package available from 1.2 version.

It is used to fetch/retrieve the elements from the Collection in forward direction only because it is also a cursor. It is a universal cursor.

It is using private inner class i.e Itr class implements from Iterator<E>

public Iterator<E> iterator();

Example :

Iterator itr = listOfCity.iterator();

Now, Iterator interface has provided two methods

public boolean hasNext() :-
It will verify, the element is available in the current position or not, if available it will return true otherwise it will return false.

public Object next() :- It will return the collection object and move the cursor to the next element object.

ListIterator<E> interface :

It is a predefined interface available in java.util package and it is the sub interface of Iterator available from JDK 1.2v.

It is used to retrieve the Collection object in both the direction i.e in forward direction as well as in backward direction. Here the inner class name is ListItr class extends from Itr class implements from ListIteartor interface

It works on the basis of index. [Will work only for List interface]

public ListIterator listIterator();

Example :

ListIterator lit = listOfCity.listIterator();

1) public boolean hasNext()
2) public Object next()
3) public boolean hasPrevious()
4) public Object previous ()

Note :- Apart from these 4 methods we have add(), set() and remove(), nextIndex() and previousIndex() method in ListIterartor interface.

forEach(Consumer<T> cons) :

forEach() method is available from JDK 1.8V.

This forEach() method is used iterate the elements from the source.

The forEach(Consumer<T> cons) method is available in java.lang.Iterable interface, It is added as a default method inside Iterable interface.

forEach(Consumer<T> cons) method of Iterable interface internally uses for each loop.

How forEach(Consumer<T>) method works internally ?

Case 1:

By using Anonymous inner class :

```
package com.ravi.collection_retrieval_demo;

import java.util.Vector;
import java.util.function.Consumer;

public class ForEachMethodInternalWorkingDemo1 {

    public static void main(String[] args)
    {
        Vector<String> listOfCity = new Vector<>();
        listOfCity.add("Hyderabad");
        listOfCity.add("Vizag");
        listOfCity.add("Pune");
        listOfCity.add("Mumbai");

        //Anonymous inner class
        Consumer<String> cons = new Consumer<String>()
         {
                @Override
                public void accept(String city)
                {
                  IO.println(city);
                }
         };

         listOfCity.forEach(cons);

    }

}
```

Case 2 :

By Using Lambda Expression :

```
package com.ravi.collection_retrieval_demo;

import java.util.Vector;
import java.util.function.Consumer;

public class ForEachMethodInternalWorkingDemo2 {

    public static void main(String[] args)
    {
        Vector<String> listOfCity = new Vector<>();
        listOfCity.add("Hyderabad");
        listOfCity.add("Vizag");
        listOfCity.add("Pune");
        listOfCity.add("Mumbai");


        /*Lambda Expression
        Consumer<String> cons =  city  -> IO.println(city.toUpperCase());
        listOfCity.forEach(cons);
        */

        //Passing Lambda directly to forEach() method
        listOfCity.forEach(city  -> IO.println(city.toUpperCase()));


    }
}
```

8) Spliterator<E> interface :

It is a predefined interface available in java.util package from java 1.8 version.

It is a cursor through which we can fetch the elements from the Collection [Collection, array, Stream]

*It is the combination of hasNext() and next() method for Iterator interface as a default method.

It is using forEachRemaining(Consumer<T> cons) method for fetching the elements which is internally using do-while loop

9) By using Method Reference : [Introduction Only]

* It is an improvement over lambda expression, By using method reference we can refer any existing method available in the our project OR java library.

```
//Program to fetch the Collection Object by using 9 ways :
---------------------------------------------------------------
package com.ravi.collection_retrieval_demo;

import java.util.Enumeration;
import java.util.Iterator;
import java.util.ListIterator;
import java.util.Spliterator;
import java.util.Vector;

public class PossibleWaysToRetrieveCollectionObjects
{
  public static void main(String[] args)
  {
      Vector<String> listOfCity = new Vector<>();
      listOfCity.add("Hyderabad");
      listOfCity.add("Vizag");
      listOfCity.add("Pune");
      listOfCity.add("Mumbai");

      IO.println("1) By using toString() Method :");
      IO.println(listOfCity.toString());

      IO.println("2) By using Ordinary for loop :");

      for(int i=0; i<listOfCity.size(); i++)
      {
          IO.println(listOfCity.get(i));
      }

      IO.println("3) By using for each loop :");

      for(String city : listOfCity)
      {
          IO.println(city);

      }

      IO.println("4) By using Enumeration<E> interface :");
      Enumeration<String> ele = listOfCity.elements();

      while(ele.hasMoreElements())
      {
          IO.println(ele.nextElement());
      }


      IO.println("5) By using Iterator<E> interface :");

      Iterator<String> itr = listOfCity.iterator();
      itr.forEachRemaining(city ->IO.println(city)); //JDK 1.8
      /*while(itr.hasNext())
      {
          IO.println(itr.next());
      }*/

      IO.println("6) By using ListIterator<E> interface :");
      ListIterator<String> lstItr = listOfCity.listIterator();

      IO.println("IN FORWARD DIRECTION :");
      while(lstItr.hasNext())
      {
          IO.println(lstItr.next());
      }

      IO.println("IN BACKWARD DIRECTION :");
      while(lstItr.hasPrevious())
      {
          IO.println(lstItr.previous());
      }


      IO.println("7) By using forEach(Consumer<T> cons) method :");
      listOfCity.forEach(city ->IO.println(city));


      IO.println("8) By using Spliterator interface :");

      Spliterator<String> spliterator = listOfCity.spliterator();
      spliterator.forEachRemaining(city ->IO.println(city));


      IO.println("9) By using Method Reference :");
      listOfCity.forEach(IO::println);

  }
}
```

Working with List interface Specific classes :

As we know, in List interface we have 4 implemented classes which are as follows :

1) Vector<E>
2) Stack<E>
3) ArrayList<E>
4) LinkedList<E>

Vector<E> :

public class Vector<E> extends AbstractList<E> implements List<E>, Cloneable, Serializable, RandomAccess

* It is an implemented legacy class of List<E> interface available from JDK 1.0V.
* It can accept duplicate, null, homogeneous and hetreogeneous objects.
* It stores the Objects based on the index position.
* It internally uses **"Dynamic Array" data structure.**
* It is dynamically Growable in nature.
* Default capacity is 10, next capacity would be 20.
* Methods are synchronized so, performance wise Vector is not good.
* Methods are synchronized so, provides thread-safety.
* Iterator is **Fail Fast iterator.**
* hashCode() & equals(Object obj) methods are overridden.
* It is mainly used to provide retrieve operation in thread-safe environment.
* It implements from Serializable, RandomAccess, Cloneable and List<E>

Constructors :

We have 4 types of Constructors in Vector

1) Vector v1 = new Vector();
   It will create the vector object with default capacity is 10.
   [Internally Object array]

2) Vector v2 = new Vector(int initialCapacity);
   Will create the vector object with user specified capacity.

3) Vector v3 = new Vector(int initialCapacity, int capacityIncrement);
   Eg :- Vector v = new Vector(100,5);

   Initially It will create the Vector Object with initial capacity 100 and then when the capacity will be full then increment by 5 so the next capacity would be 105, 110 and so on.

4) Vector v4 = new Vector(Collection c);
   We can achieve loose coupling

//Programs on Vector :

```
package com.ravi.vector_demo;

import java.util.Vector;

//add all the elements of Vector

public class VectorDemo1
{
    public static void main(String[] args)
    {
        Vector<Integer> listOfNumbers = new Vector<>();
        listOfNumbers.add(100);
        listOfNumbers.add(200);
        listOfNumbers.add(300);
        listOfNumbers.add(400);
        listOfNumbers.add(500);

        Integer sum = 0;

        for(Integer number : listOfNumbers)
        {
            sum += number;
        }

        IO.println("Sum of Vector elements :"+sum);

    }

}
```

```
package com.ravi.vector_demo;

import java.util.Collections;
import java.util.Vector;

//Custom Object

record Product(Integer id, String name, Double price) implements Comparable<Product>
{
    @Override
    public int compareTo(Product p2)
    {
        return Double.compare(this.price(), p2.price());
    }
}


public class VectorDemo2
{
    public static void main(String[] args)
    {
     Vector<Product> listOfProducts = new Vector<>();
     listOfProducts.add(new Product(333, "Camera", 25000D));
     listOfProducts.add(new Product(111, "Mobile", 54000D));
     listOfProducts.add(new Product(222, "Laptop", 96000D));

     IO.println("Original Data :");
     for(Product product : listOfProducts)
     {
         IO.println(product);
     }

     Collections.sort(listOfProducts);

     IO.println("Sorted Product Object based on the Price :");
     for(Product product : listOfProducts)
     {
         IO.println(product);
     }


    }
}
```

```
package com.ravi.vector_demo;

import java.util.Vector;

//Vector Constructor
public class VectorDemo3 {

    public static void main(String[] args)
    {
        Vector<Integer> listOfNumber = new Vector<>(100, 20);

        IO.println("Initial Capacity is :"+listOfNumber.capacity());

        for(int i=0; i<100; i++)
        {
            listOfNumber.add(i);
        }

        IO.println("After adding 100 elements :");
        IO.println("Size is  :"+listOfNumber.size());
        IO.println("Capacity is  :"+listOfNumber.capacity());

        listOfNumber.add(999);
        IO.println("After adding 999 element :");
        IO.println("Size is  :"+listOfNumber.size());
        IO.println("Capacity is  :"+listOfNumber.capacity());
    }

}
```

---

public default sort(Comparator<T> cons)

* A default method added inside List<E> interface from JDK 1.8V to provide backward compatibility.

* It accepts Comparator<T> as a parameter so we can provide custom sorting.

```
package com.ravi.vector_demo;

import java.util.Collections;
import java.util.Vector;

public class VectorDemo4 {

    public static void main(String[] args)
    {
        Vector<Integer> listOfNumber = new Vector<>();
        listOfNumber.add(67);
        listOfNumber.add(34);
        listOfNumber.add(99);
        listOfNumber.add(12);
        listOfNumber.add(29);

        IO.println("Original Data :"+listOfNumber);

        IO.println();

        Collections.sort(listOfNumber);
        IO.println("Sorting in Ascending Order :"+listOfNumber);

        /*Old Approach
        Collections.sort(listOfNumber, (i1,i2)-> i2.compareTo(i1));
        IO.println("Sorting in Descending Order :"+listOfNumber);
        */

        listOfNumber.sort((i1,i2)-> Integer.compare(i2, i1));
        IO.println("Sorting in Descending Order :"+listOfNumber);

        IO.println("Maximum element is :"+Collections.max(listOfNumber));
        IO.println("Minimum element is :"+Collections.min(listOfNumber));

    }

}
```

Collections.max() & Collections.min() both are accepting Collection interface as a parameter.